

Name: fetalino jayvee m.	Date Performed: 26/02/2026
Course/Section: CPE31S1	Date Submitted: 26/02/2026
Instructor: ENGR. ROBIN VALENZUELA	Semester and SY:
Activity 7: Managing Files and Creating Roles in Ansible	
1. Objectives: 1.1 Manage files in remote servers 1.2 Implement roles in ansible	
2. Discussion: In this activity, we look at the concept of copying a file to a server. We are going to create a file into our git repository and use Ansible to grab that file and put it into a particular place so that we could do things like customize a default website, or maybe install a default configuration file. We will also implement roles to consolidate plays.	
Task 1: Create a file and copy it to remote servers - Using the previous directory we created, create a directory, and named it "files." Create a file inside that directory and name it "default_site.html." Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit. - Edit the site.yml file and just below the web_servers play, create a new file to copy the default html file for site: - name: copy default html file for site - tags: apache, apache2, httpd - copy: - src: default_site.html - dest: /var/www/html/index.html - owner: root - group: root - mode: 0644 - Run the playbook site.yml. Describe the changes.	

The screenshot shows the VS Code editor interface. The Explorer sidebar on the left displays a project structure with folders 'HOA' and 'files', and files 'dbservers', 'fileservers', 'webservers', 'ansible.cfg', 'inventory.ini', 'playbook.yml', and 'site.yml'. The 'default_site.html' file is selected in the Explorer and is also open in the editor. The editor displays the content of 'default_site.html', which is an HTML file with the following content:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <title>Page Title</title>
6 </head>
7 <body>
8
9   <h1>This is a Heading</h1>
10  <p>This is a paragraph.</p>
11
12 </body>
13 </html>
14
```

The terminal at the bottom shows the execution of the Ansible playbook 'site.yml' on the host 'zerojay@zerojay'. The output indicates that the task 'install apache and php for CentOS servers' was skipped for the host 192.168.56.102. The PLAY RECAP shows that the host 192.168.56.102 was ok=4, changed=1, unreachable=0, failed=0, and skipped=1. The host 192.168.56.103 was ok=1, changed=0, unreachable=0, failed=0, and skipped=0. The host 192.168.56.104 was ok=1, changed=0, unreachable=0, failed=0, and skipped=0.

The screenshot shows a remote server terminal window titled 'zerojay@zerojay:~'. The terminal displays the following commands and output:

```
zerojay@zerojay:~$ cat index.html
cat: index.html: No such file or directory
zerojay@zerojay:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
zerojay@zerojay:~$ cat /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Page Title</title>
</head>
<body>
  <h1>This is a Heading</h1>
  <p>This is a paragraph.</p>
</body>
</html>
zerojay@zerojay:~$
```

4. Go to the remote servers (**web_servers**) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (**default_site.html**). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output. Sync your local repository with GitHub and describe the changes.

The file that I made in default_size.html, the contents inside it was copied in my web server /var/www/html/index.html

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:

- hosts: workstations
become: true
tasks:
 - name: install unzip
package:
name: unzip
 - name: install terraform
unarchive:

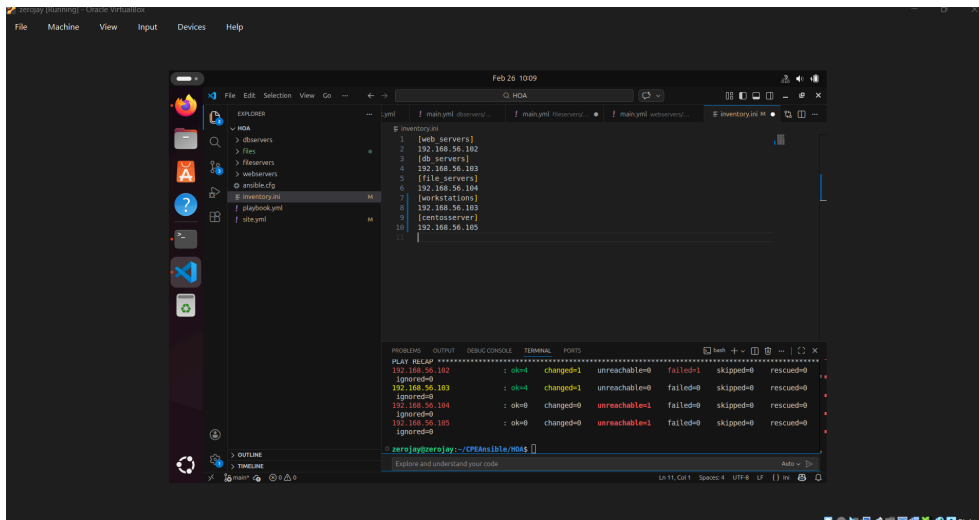
src:

https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip

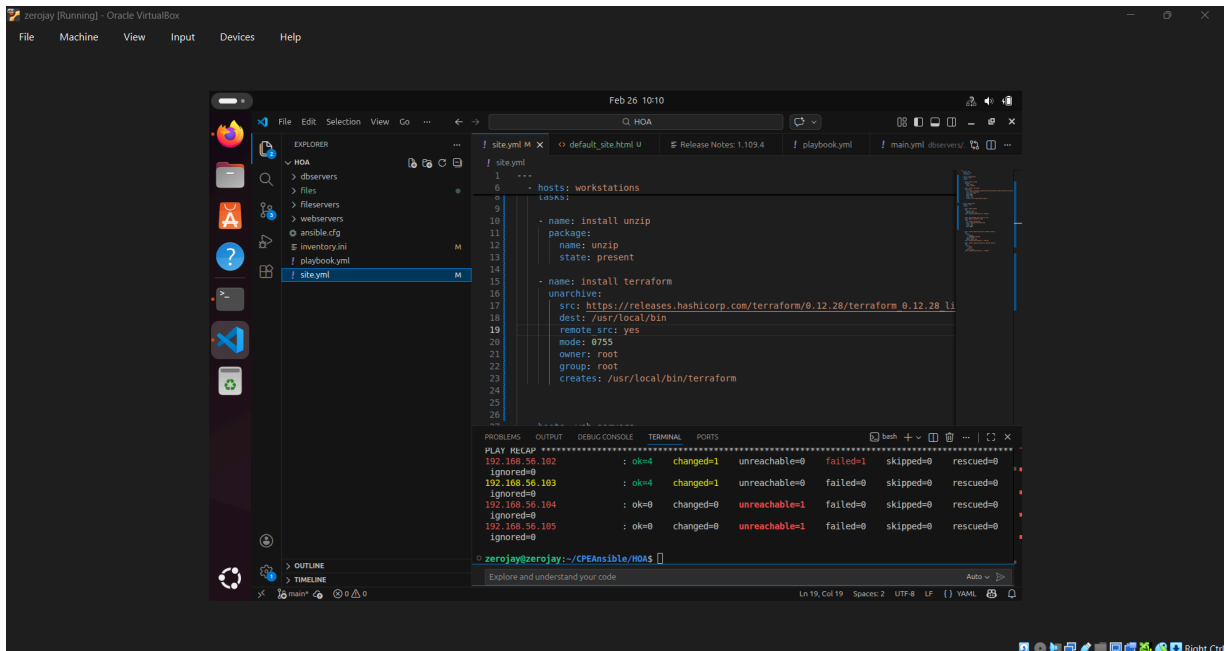
dest: /usr/local/bin
remote_src: yes
mode: 0755
owner: root
group: root

2. Edit the inventory file and add workstations group. Add any Ubuntu remote server. Make sure to remember the IP address.

new Inventory/ini

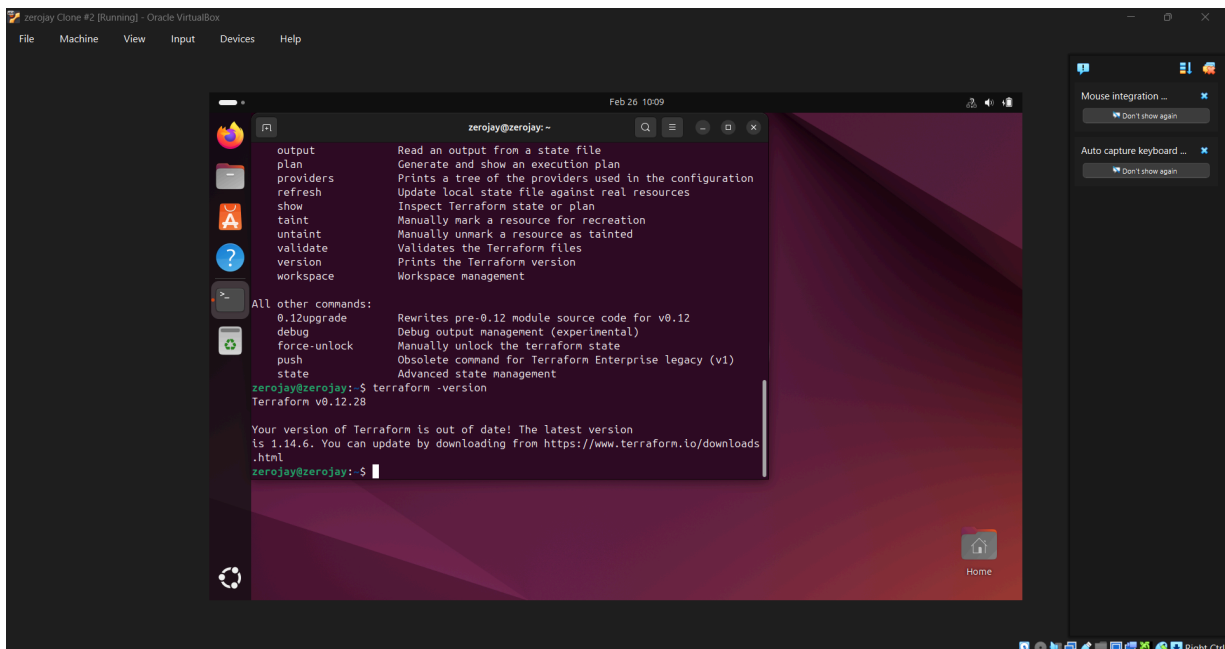


3. Run the playbook. Describe the output.
it smoothly runs when I tried executing it, just some minor errors like unreachable because some of my managed nodes is off



4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

terraform in my clone



Task 3: Create roles

1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)

```
---
- hosts: all
  become: true
  pre_tasks:

    - name: update repository index (CentOS)
      tags: always
      dnf:
        update_cache: yes
        changed_when: false
        when: ansible_distribution == "CentOS"
    - name: install updates (Ubuntu)
      tags: always
      apt:
        update_cache: yes
        changed_when: false
        when: ansible_distribution == "Ubuntu"

- hosts: all
  become: true
  roles:
    - base

- hosts: workstations
  become: true
  roles:
    - workstations

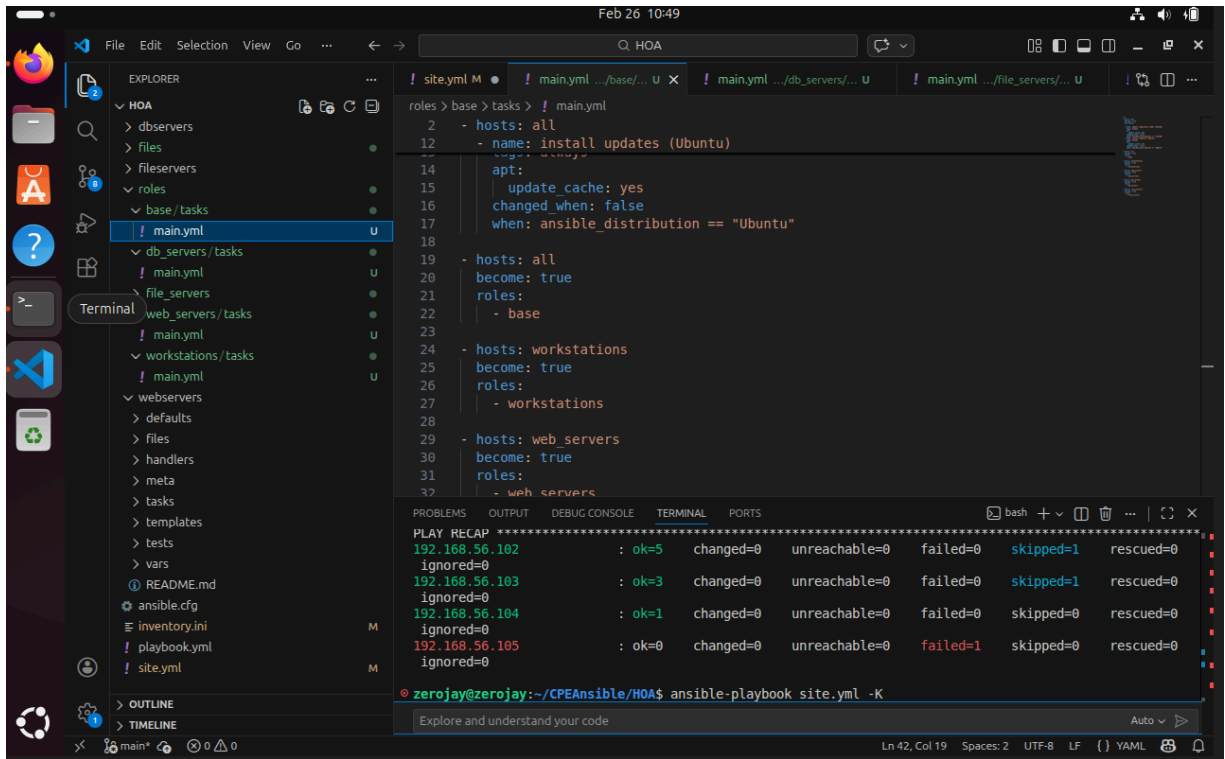
- hosts: web_servers
  become: true
  roles:
    - web_servers

- hosts: db_servers
  become: true
  roles:
    - db_servers

- hosts: file_servers
  become: true
  roles:
    - file_servers
```

Save the file and exit.

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a directory structure for an Ansible project. The terminal displays the output of an Ansible playbook run, including a recap of tasks and their results.

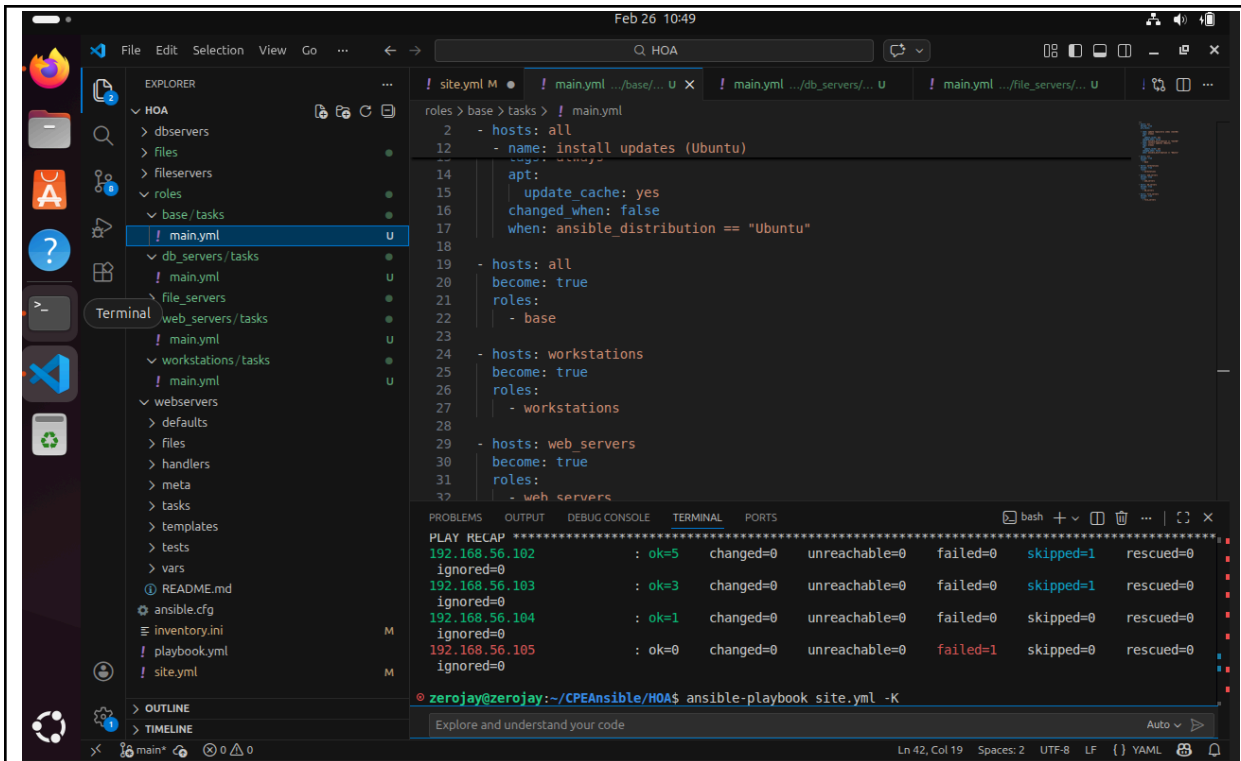
```
roles > base > tasks > ! main.yml
2  - hosts: all
12 - name: install updates (Ubuntu)
14   apt:
15     update_cache: yes
16     changed_when: false
17     when: ansible_distribution == "Ubuntu"
18
19 - hosts: all
20   become: true
21   roles:
22     - base
23
24 - hosts: workstations
25   become: true
26   roles:
27     - workstations
28
29 - hosts: web_servers
30   become: true
31   roles:
32     - web_servers
```

PLAY RECAP

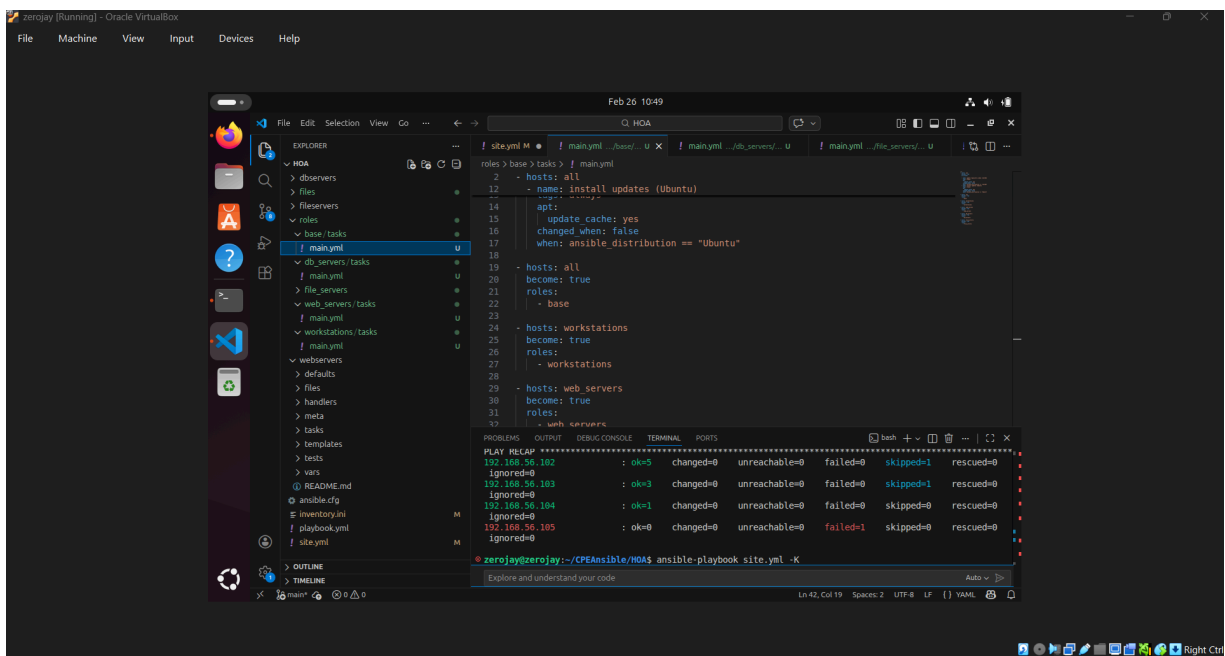
Host	ok	changed	unreachable	failed	skipped	rescued
192.168.56.102	5	0	0	0	1	0
192.168.56.103	3	0	0	0	1	0
192.168.56.104	1	0	0	0	0	0
192.168.56.105	0	0	0	1	0	0

zerojay@zerojay:~/CPEAnsible/HOA\$ ansible-playbook site.yml -K

3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.



4. Run the site.yml playbook and describe the output.



the other one failed because I have different become password so I retried it with its become password and here is the outcome

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PLAY RECAP *****
192.168.56.102 : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
192.168.56.103 : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
192.168.56.104 : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
192.168.56.105 : ok=2    changed=0    unreachable=0    failed=1    skipped=1    rescued=0
ignored=0
zerojay@zerojay:~/CPEAnsible/HOA$
```

now its the only one with OK but the other three failed because the difference in become password

Reflections:

Answer the following:

1. What is the importance of creating roles?

Roles help organize Ansible tasks into structured and reusable components. They make playbooks cleaner, easier to manage, and more scalable. Roles also allow configurations to be reused across different servers or projects, improving consistency and reducing errors.

2. What is the importance of managing files?

Managing files ensures that all remote servers have consistent configurations and content. It automates file deployment, maintains correct permissions, and prevents configuration differences between servers, making system administration more efficient and reliable.